

Übungsprotokoll

INSY– Informationssysteme



Übungsdatum:
3/11/25 – 04/12/25

Klasse:
3BHIT

Name:
Kornberger Adrian

Abgabedatum:
04.12.2025

Gruppe:
INSY2

Note:

Leitung:
Dipl.-Ing.(FH) MESTL
Alexander

Mitübende:

Übungsbezeichnung:

Arbeiten mit SQL

Inhaltsverzeichnis:

1	Aufgabenstellung.....	2
2	Abstract (English).....	2
3	Theoretische Grundlagen	2
4	Übungsdurchführung.....	2
5	Ergebnisse.....	11
6	Code	12
7	Kommentar	12

1 Aufgabenstellung

Wir werden in einer virtuellen Maschine MariaDB (Fork von MySQL) installieren, Daten mithilfe des angefügten Scripts importieren und SQL-Beispiele aus dem Aufgabendokument "insyExercise3.pdf" (in den "Dateien" dieses Teams) durchführen.

2 Abstract (English)

This collection of SQL exercises introduces fundamental concepts of relational data processing, including data aggregation, join operations, and grouping techniques. It demonstrates how Inner Joins retrieve only matching records, while Left Joins preserve all rows from the left table regardless of matching entries. The tasks also illustrate how Group By is used to create aggregated summaries and how the Having clause filters groups based on aggregate conditions. Through practical examples with employees, departments, locations, and countries, the exercises highlight essential query patterns for counting, filtering, and combining data across multiple tables.

3 Theoretische Grundlagen

Inner Join: nur passende Zeilen aus beiden Tabellen.

Left Join: alle Zeilen links, rechts ggf. NULL.

Group By: gruppiert Zeilen für Zusammenfassungen.

Aggregation: z. B. COUNT, SUM über Gruppen.

Having: Filter nach der Gruppierung.

Kardinalität: 1:1, 1:n beeinflusst Join-Ergebniszeilen.

4 Übungsdurchführung

Bsp 1.1.1

SELECT first_name, last_name, salary, email FROM EMPLOYEES ORDER BY last_name, first_name;

	first_name	last_name	salary	email
1	Ellen	Abel	11000.00	EABEL
2	Sundar	Ande	6400.00	SANDE
3	Mozhe	Atkinson	2800.00	MATKINSO
4	David	Austin	4800.00	DAUSTIN
5	Hermann	Baer	10000.00	HBAER
6	Shelli	Baida	2900.00	SBAIDA
7	Amit	Banda	6200.00	ABANDA
8	Elizabeth	Bates	7300.00	EBATES
9	Sarah	Bell	4000.00	SBELL
10	David	Bernstein	9500.00	DBERNSTE
11	Laura	Bissot	3300.00	LBISSOT
12	Harrison	Bloom	10000.00	HBLOOM

Bsp 1.1.2

SELECT first_name || ' ' || last_name || ', ' || concat(salary) || ', ' || email AS "employee data"
FROM EMPLOYEES
ORDER BY last_name, first_name;

	'employee data'
1	1
2	1
3	1
4	1
5	1
6	1
7	1
8	1
9	1
10	1
11	1
12	1
13	1

Bsp 1.1.3

SELECT first_name, last_name, job_id, salary FROM EMPLOYEES WHERE job_id = 'IT_PROG' AND salary > 12000 ORDER BY first_name, last_name;

	first_name	last_name	job_id	salary
1	Adrian	Kornberger	IT_PROG	600000.00
2	Michael	Pointner	IT_PROG	60000.00

Bsp 1.1.4

SELECT first_name, last_name, department_id FROM EMPLOYEES WHERE last_name LIKE 'H%' AND last_name NOT LIKE '%r';

	first_name	last_name	department_id
1	Alexander	Hunold	60
2	Guy	Himuro	30
3	Peter	Hall	80
4	Alyssa	Hutton	80
5	Michael	Hartstein	20
6	Shelley	Higgins	110

Bsp. 1.1.5

SELECT last_name, first_name FROM EMPLOYEES WHERE job_id IN ('SA_MAN','SA_REP','ST_CLERK') AND job_id <> 'IT_PROG';

	last_name	first_name
1	Nayer	Julia
2	Mikkilineni	Irene
3	Landry	James
4	Markle	Steven
5	Bissot	Laura
6	Atkinson	Mozhe
7	Marlow	James
8	Olson	TJ
9	Mallin	Jason
10	Rogers	Michael
11	Gee	Ki
12	Philtanker	Hazel
13	Ladwig	Renske

Bsp 1.1.7

SELECT * FROM EMPLOYEES WHERE salary BETWEEN 10000 AND 17000;

EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMISSION
1	Neena	Kochhar	NKOCHHAR	515.123.4568	2005-09-21	AD_VP	17000.00	
2	Lex	De Haan	LDEHAAN	515.123.4569	2001-01-13	AD_VP	17000.00	
3	Nancy	Greenberg	NGREENBE	515.124.4569	2002-08-17	FI_MGR	12008.00	
4	Den	Raphaely	DRAPHEAL	515.127.4561	2002-12-07	PU_MAN	11000.00	
5	John	Russell	JRUSSEL	011.44.1344.429268	2004-10-01	SA_MAN	14000.00	
6	Karen	Partners	KPARTNER	011.44.1344.467268	2005-01-05	SA_MAN	13500.00	
7	Alberto	Errazuriz	AERRAZUR	011.44.1344.429278	2005-03-10	SA_MAN	12000.00	
8	Gerald	Cambraut	GCAMBRAU	011.44.1344.619268	2007-10-15	SA_MAN	11000.00	
9	Eleni	Zlotkey	EZLOTKEY	011.44.1344.429018	2008-01-29	SA_MAN	10500.00	
10	Peter	Tucker	PTUCKER	011.44.1344.129268	2005-01-30	SA_REP	10000.00	
11	Janette	King	JKING	011.44.1345.429268	2004-01-30	SA_REP	10000.00	
12	Clara	Vishney	CVISHNEY	011.44.1346.129268	2005-11-11	SA_REP	10500.00	
13	Lisa	Ozer	LOZER	011.44.1344.129268	2005-03-11	SA_REP	11500.00	

Bsp 1.1.8

SELECT first_name, last_name, hire_date FROM EMPLOYEES WHERE hire_date <= ADD_MONTHS(SYSDATE(), -60);

	first_name	last_name	hire_date
1	Steven	King	2003-06-17
2	Neena	Kochhar	2005-09-21
3	Lex	De Haan	2001-01-13
4	Alexander	Hunold	2006-01-03
5	Bruce	Ernst	2007-05-21
6	David	Austin	2005-06-25
7	Valli	Pataballa	2006-02-05
8	Diana	Lorentz	2007-02-07
9	Nancy	Greenberg	2002-08-17
10	Daniel	Faviet	2002-08-16
11	John	Chen	2005-09-28
12	Ismael	Sciarra	2005-09-30
13	Jose Manuel	Urman	2006-03-07

Bsp 1.1.9

SELECT first_name, last_name FROM EMPLOYEES ORDER BY first_name DESC, last_name FETCH FIRST 10 ROWS ONLY;

	first_name	last_name
1	Winston	Taylor
2	William	Gietz
3	William	Smith
4	Vance	Jones
5	Valli	Pataballa
6	Trenna	Rajs
7	TJ	Olson
8	Timothy	Gates
9	Tayler	Fox
10	Susan	Mavris

Bsp 1.1.10

SELECT FIRST_NAME, LAST_NAME FROM EMPLOYEES ORDER BY employee_id LIMIT 21 OFFSET 89;

	FIRST_NAME	LAST_NAME
1	Jennifer	Dilly
2	Timothy	Gates
3	Randall	Perkins
4	Sarah	Bell
5	Britney	Everett
6	Samuel	McCain
7	Vance	Jones
8	Alana	Walsh
9	Kevin	Feeney
10	Donald	OConnell
11	Douglas	Grant
12	Jennifer	Whalen
13	Michael	Hartstein

Bsp 1.2.1

Datenaggregation bezeichnet den Vorgang, bei dem große Mengen von Rohdaten zusammengefasst, verdichtet oder gruppiert werden, um übersichtliche Informationen zu erhalten.

Bsp 1.2.2

```
SELECT e.first_name, e.last_name, d.department_name, c.country_name FROM EMPLOYEES e JOIN
DEPARTMENTS d ON e.department_id = d.department_id JOIN LOCATIONS l ON d.location_id =
l.location_id JOIN COUNTRIES c ON l.country_id = c.country_id WHERE d.department_name = 'IT';
```

first_name	last_name	department_name	country_name
Alexander	Hunold	IT	United States of America
Bruce	Ernst	IT	United States of America
David	Austin	IT	United States of America
Valli	Pataballa	IT	United States of America
Diana	Lorentz	IT	United States of America
Adrian	Kornberger	IT	United States of America
Kristijan	Vojonowatch	IT	United States of America
Michael	Pointner	IT	United States of America

Bsp 1.2.3

A INNER JOIN B ist gleich B INNER JOIN A. Die Zeilenmenge ist die gleiche, nur die Spaltenreihenfolge kann anders sein. In Mengenalgebra entspricht der Inner Join der Schnittmenge nach passenden Tupeln.

Bsp 1.2.4

Das Ergebnis hat gleich viele Zeilen wie A, wenn jede Zeile in A genau ein passendes Gegenstück in B hat, weniger Zeilen als A, wenn einige A-Zeilen kein Match in B haben und daher ausgeschlossen werden, und mehr Zeilen als A, wenn manche A-Zeilen zu mehreren Zeilen in B passen.

Bsp 1.2.5

Gegeben sind A mit 5 Zeilen und B mit 10 Zeilen. Ein A CROSS JOIN B ergibt 5 mal 10 = 50 Zeilen.

Bsp 1.2.6

```
SELECT e.first_name, e.last_name, COALESCE(d.department_name, 'no department') AS  
department_name FROM EMPLOYEES e LEFT JOIN DEPARTMENTS d ON e.department_id =  
d.department_id;
```

	first_name	last_name	department_name
1	Steven	King	Executive
2	Neena	Kochhar	Executive
3	Lex	De Haan	Executive
4	Alexander	Hunold	IT
5	Bruce	Ernst	IT
6	David	Austin	IT
7	Valli	Pataballa	IT
8	Diana	Lorentz	IT
9	Nancy	Greenberg	Finance
10	Daniel	Faviet	Finance
11	John	Chen	Finance
12	Ismael	Sciarra	Finance
13	Jose Manuel	Urman	Finance

Bsp 1.2.7

A LEFT JOIN B ist nicht gleich B LEFT JOIN A. Left Join ist nicht kommutativ, da die linke Tabelle bestimmt, welche Zeilen auch ohne passendes Gegenstück erhalten bleiben.

Bsp1.2.8

```
SELECT e.first_name, e.last_name, COALESCE(d.department_name, 'no department') AS  
department_name FROM EMPLOYEES e LEFT JOIN DEPARTMENTS d ON e.department_id =  
d.department_id;
```

	first_name	last_name	department_name
1	Steven	King	Executive
2	Neena	Kochhar	Executive
3	Lex	De Haan	Executive
4	Alexander	Hunold	IT
5	Bruce	Ernst	IT
6	David	Austin	IT
7	Valli	Pataballa	IT
8	Diana	Lorentz	IT
9	Nancy	Greenberg	Finance
10	Daniel	Faviet	Finance
11	John	Chen	Finance
12	Ismael	Sciarra	Finance
13	Jose Manuel	Urman	Finance
14	Luis	Popp	Finance

Bsp1.2.9

Nein Left Join ist nicht kommutativ.

Bsp1.2.10

Die Query verwendet nur LEFT JOINS, daher werden alle Employees angezeigt, auch wenn keine zugehörige Abteilung, Location oder Country vorhanden ist. Die Reihenfolge der Joins ist entscheidend, da jede weitere LEFT JOIN auf die vorherige Tabelle angewendet wird. Unabhängig vom Datenbestand liefert die Abfrage somit für jeden Employee mindestens eine Zeile mit NULL für fehlende Werte, sie beschreibt die Abfrage also korrekt.

Bsp 1.3.1

```
SELECT MAX(salary) AS max_salary, MIN(salary) AS min_salary FROM EMPLOYEES;
```

	max_salary	min_salary
1	600000.00	600.00

Bsp 1.3.2

```
SELECT d.department_name, MIN(e.salary) AS min_salary, MAX(e.salary) AS max_salary FROM
DEPARTMENTS d JOIN EMPLOYEES e ON d.department_id = e.department_id GROUP BY
d.department_name;
```

	department_name	min_salary	max_salary
1	Accounting	8300.00	12008.00
2	Administration	4400.00	4400.00
3	Executive	17000.00	24000.00
4	Finance	6900.00	12008.00
5	Human Resources	6500.00	6500.00
6	IT	600.00	60000.00
7	Marketing	6000.00	13000.00
8	Public Relations	10000.00	10000.00
9	Purchasing	2500.00	11000.00
10	Sales	6100.00	14000.00
11	Shipping	2100.00	8200.00

Bsp 1.3.3

```
SELECT d.department_name, COUNT(e.employee_id) AS num_employees FROM DEPARTMENTS d
LEFT JOIN EMPLOYEES e ON d.department_id = e.department_id GROUP BY d.department_name
ORDER BY num_employees DESC;
```

	department_name	num_employees
1	Shipping	45
2	Sales	34
3	IT	8
4	Purchasing	6
5	Finance	6
6	Executive	3
7	Marketing	2
8	Accounting	2
9	Administration	1
10	Public Relations	1
11	Human Resources	1
12	NOC	0
13	Payroll	0
14	Construction	0

Bsp 1.3.4

```
SELECT d.department_name, COUNT(e.employee_id) AS num_employees FROM DEPARTMENTS d
JOIN EMPLOYEES e ON d.department_id = e.department_id WHERE e.job_id <> 'IT_PROG' GROUP BY
d.department_name HAVING COUNT(e.employee_id) > 5;
```

	department_name	num_employees
1	Finance	6
2	Purchasing	6
3	Sales	34
4	Shipping	45

Bsp 1.3.5

Ja, beide Queries beschreiben dieselbe Abfrage, weil sie dieselben Gruppierungsattribute verwenden.

Bsp 1.3.6

```
SELECT c.country_name, COALESCE(COUNT(d.department_id), 0) AS dept_count FROM COUNTRIES c
LEFT JOIN LOCATIONS l ON c.country_id = l.country_id LEFT JOIN DEPARTMENTS d ON l.location_id =
d.location_id GROUP BY c.country_name;
```

13	Italy	0
14	Japan	0
15	Kuwait	0
16	Malaysia	0
17	Mexico	0
18	Netherlands	0
19	Nigeria	0
20	Singapore	0
21	Switzerland	0
22	United Kingdom	2
23	United States of America	23
24	Zambia	0
25	Zimbabwe	0

Bsp 1.3.7

```
SELECT c.country_id, COUNT(d.department_id) AS num_departments FROM COUNTRIES c JOIN
LOCATIONS l ON c.country_id = l.country_id JOIN DEPARTMENTS d ON l.location_id = d.location_id
WHERE d.department_name NOT LIKE '%Sales%'
```

Beispiel: Sales ausschließen GROUP BY c.country_id HAVING COUNT(d.department_id) > 3;

	country_id	num_departments
1	US	24

Bsp 1.3.8

```
SELECT c.country_id, COUNT(d.department_id) AS dept_count FROM COUNTRIES c LEFT JOIN
LOCATIONS l ON c.country_id = l.country_id LEFT JOIN DEPARTMENTS d ON l.location_id =
d.location_id GROUP BY c.country_id;
```

	country_id	dept_count
9	DK	0
10	EG	0
11	FR	0
12	IL	0
13	IN	0
14	IT	0
15	JP	0
16	KW	0
17	ML	0
18	MX	0
19	NG	0
20	NL	0
21	SG	0
22	UK	2
23	US	23
24	ZM	0
25	ZW	0

5 Ergebnisse

6 Code

7 Kommentar

